

Architecturer des équipes d'agents pour les tâches complexes

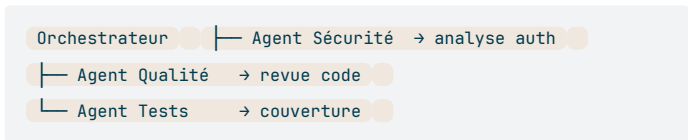
Quand passer en multi-agent

Un seul agent reste optimal jusqu'à environ 7 répertoires ou 50 fichiers. Au-delà, la fenêtre de contexte se remplit à 80-90% simplement pour charger les fichiers pertinents, ce qui ne laisse presque plus de place au raisonnement. Répartir le travail entre plusieurs agents maintient chacun autour de 40% d'utilisation, avec suffisamment d'espace pour analyser et décider.

Scope	Agent unique	Équipe
<10 répertoires	Confortable (~30%)	Inutile
50K lignes	Dégradé (80-90%)	Recommandé
100K+ lignes	Context overflow	Indispensable

Topologie étoile (Star)

L'orchestrateur central reçoit l'objectif global, le décompose en sous-tâches, délègue à des agents spécialisés, puis synthétise leurs résultats. C'est la topologie par défaut de Claude Code Agent Teams.



L'orchestrateur planifie et délègue ; il ne code pas lui-même. Son rôle est d'allouer le travail, de débloquer les dépendances, et de présenter une réponse unifiée.

Topologie pipeline

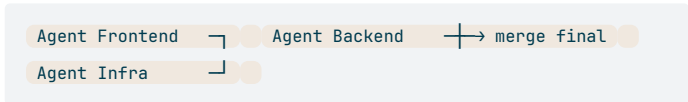
Les agents travaillent en séquence : la sortie de l'un devient l'entrée du suivant. Utile quand les étapes sont strictement ordonnées et que chaque résultat conditionne le suivant.



Avantage : chaque agent se concentre sur une phase précise, sans bruit des phases précédentes dans son contexte.
Inconvénient : un blocage sur une étape arrête tout le pipeline.

Topologie parallèle

Des agents indépendants travaillent simultanément sur des modules sans dépendance entre eux. Cas typique : refactoring frontend, backend et infra en parallèle sur des fichiers non partagés.



Condition d'applicabilité : les modules doivent avoir zéro état partagé. Si les agents doivent constamment lire les sorties des autres ou modifier des fichiers communs, les conflits git et les messages de coordination annulent le gain.

Isolation et permissions par agent

Chaque agent dispose de sa propre fenêtre de contexte (1M tokens avec Opus 4.6) et peut recevoir une whitelist d'outils distincte. Limiter un agent aux outils `Read`, `Glob`, `Grep` empêche toute modification accidentelle pendant la phase d'analyse.

Agent	Outils autorisés
Explorateur	Read, Glob, Grep
Planificateur	Read, Glob, Grep, Write(plan)

Agent	Outils autorisés
Implémenteur	Read, Edit, Bash, Write
Relecteur	Read, Glob, Grep

Activer les Agent Teams

```
# Variable d'environnement (session courante) export
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS = 1 claude
# Configuration persistante ~/.claude/settings.json {
  "env" : {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS" : "1"
  }
}
```

Prérequis : Claude Code v2.1.32+, modèle Opus 4.6 (`/model opus`), dépôt git initialisé. Navigation entre agents via `Shift+Down` en mode in-process.

Guardrails (v3.38.0)

Trois contrôles indispensables pour les équipes autonomes :

Boucles — `MAX_ITERATIONS=8` et un prompt de réflexion obligatoire avant chaque cycle. Si l'agent atteint la limite, tuer ou réassigner plutôt que de laisser tourner.

Reviewer dédié — 1 agent Opus 4.6 read-only (outils `Read/Grep/Glob`) auto-déclenché sur `TaskCompleted` , pour 4 agents workers. Ratio 1:4. (Crédit : Addy Osmani)

Budget tokens par agent — Fixer une limite stricte de tokens par agent ; pause automatique à 85% pour vérification humaine avant de continuer.

Règle du >5 agents

Démarrer avec 2 à 3 agents, puis augmenter progressivement. Au-delà de 5, le coût de coordination (messages mailbox, conflits git) dépasse généralement le gain de parallélisation. L'exception : modules physiquement indépendants sur une base de code de 100K+ lignes.